

Scientific Machine Learning Final Project

Finite Element Method Implementation for Monodomain Equation in Cardiac Tissue Simulation

Paolo Deidda, Raffaele Perri

June 2025

Contents

1	Theoretical Formulations	3
1.1	IMEX Time Integration Scheme	3
1.2	Weak Formulation	3
1.3	Finite Element Discretization	4
1.4	Modified assembleDiffusion Function	5
1.5	MATLAB code that solves the problem stated at point 2.	5
1.6	Set the diffusivity $\Sigma_d \in \{10\Sigma_h, \Sigma_h, 0.1\Sigma_h\}$ in Ω_d	5
1.7	Graphic visualization of the significant results	6
2	Ottimizzazione del modello DeepRitz per l'equazione del monodominio	7
2.1	Introduzione all'approccio DeepRitz	7
2.2	Architettura iniziale e limitazioni	8
2.3	Ottimizzazione dell'architettura di rete	8
2.3.1	Incremento della profondità e capacità	8
2.4	Revisione della condizione iniziale	9
2.4.1	Da gradino a funzione gaussiana	9
2.5	Bilanciamento dei pesi della loss	9
2.5.1	Analisi delle singole componenti	9
2.5.2	Riconfigurazione progressiva	9
2.6	Strategie di campionamento avanzate	10
2.6.1	Incremento del numero di punti	10
2.6.2	Rigenerazione dei punti ad ogni epoca	10
2.7	Ottimizzazione dell'ottimizzatore	10
2.7.1	Learning rate e scheduler	11
2.7.2	Implementazione di checkpointing completo	11
2.8	Normalizzazione dell'input	12
2.9	Risultati progressivi ottenuti	12
2.9.1	Andamento della loss	12
2.9.2	Qualità della propagazione dell'onda	12
2.10	Confronto con FEM e approcci alternativi	13
2.11	Lezioni apprese e conclusioni	13

3	Deep Learning Approaches	13
3.1	Convolutional Neural Network (CNN) as a Surrogate Model	14
3.1.1	Initial Model: <i>cnn_model_000001</i>	14
3.1.2	Advanced Model: <i>cnn_model_000002</i>	14
3.1.3	Further Experiments: <i>cnn_model_010647</i> and <i>cnn_model_104653</i> . . .	15
3.2	Deep Ritz Method	16
3.2.1	Model <i>deepritz_model_220842</i>	16
3.3	Physics-Informed Neural Network (PINN)	17
3.3.1	Model <i>pinm_model_113908</i>	17

1 Theoretical Formulations

1.1 IMEX Time Integration Scheme

To solve the monodomain equation efficiently, we use a first-order Implicit-Explicit (IMEX) scheme that treats the diffusion term implicitly and the nonlinear reaction term explicitly. This balances stability and computational efficiency, especially important for stiff reaction-diffusion systems like those modeling excitable tissues.

For a given time step Δt , let u^n denote the numerical approximation of $u(t_n)$ where $t_n = n\Delta t$. The first-order IMEX scheme can be written as:

$$\frac{u^{n+1} - u^n}{\Delta t} - \nabla \cdot \Sigma \nabla u^{n+1} + f(u^n) = 0 \quad (1)$$

Rearranging this equation to isolate the unknown u^{n+1} , we obtain:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \quad (2)$$

This can be written in operator form as:

$$(I - \Delta t \mathcal{L})u^{n+1} = u^n - \Delta t f(u^n) \quad (3)$$

where I is the identity operator and $\mathcal{L}$ is the diffusion operator defined by $\mathcal{L}u = \nabla \cdot \Sigma \nabla u$. The implicit treatment of diffusion ensures unconditional stability in space, while the explicit treatment of the reaction term avoids nonlinear solvers at each step.

1.2 Weak Formulation

Starting from the IMEX scheme:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \quad (4)$$

Multiplying by the test function v and integrating over Ω :

$$\int_{\Omega} (u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1}) v \, d\Omega = \int_{\Omega} (u^n - \Delta t f(u^n)) v \, d\Omega \quad (5)$$

The key step in the derivation is the application of integration by parts to the diffusion term. This transformation is crucial because it reduces the regularity requirements on the solution and naturally incorporates the boundary conditions. Applying integration by parts to the second term on the left-hand side:

$$\int_{\Omega} \nabla \cdot \Sigma \nabla u^{n+1} v \, d\Omega = \int_{\partial\Omega} \Sigma \frac{\partial u^{n+1}}{\partial \mathbf{n}} v \, d\Gamma - \int_{\Omega} \Sigma \nabla u^{n+1} \cdot \nabla v \, d\Omega \quad (6)$$

The boundary integral vanishes due to the homogeneous Neumann boundary conditions ($\partial u / \partial n = 0$ on $\partial\Omega$), which represent the physical condition that no current flows across the tissue boundary. This leads to the weak formulation:

Find $u^{n+1} \in H^1(\Omega)$ such that for all $v \in H^1(\Omega)$:

$$\int_{\Omega} u^{n+1} v \, d\Omega + \Delta t \int_{\Omega} \Sigma \nabla u^{n+1} \cdot \nabla v \, d\Omega = \int_{\Omega} u^n v \, d\Omega - \Delta t \int_{\Omega} f(u^n) v \, d\Omega \quad (7)$$

This weak formulation can be expressed in the standard bilinear form $a(u^{n+1}, v) = L^n(v)$, where:

$$a(u, v) = \int_{\Omega} uv \, d\Omega + \Delta t \int_{\Omega} \Sigma \nabla u \cdot \nabla v \, d\Omega \quad (8)$$

$$L^n(v) = \int_{\Omega} u^n v \, d\Omega - \Delta t \int_{\Omega} f(u^n) v \, d\Omega \quad (9)$$

This weak formulation is symmetric, coercive, and continuous—ensuring existence and uniqueness of the solution via the Lax-Milgram theorem.

1.3 Finite Element Discretization

The finite element space is defined as:

$$V_h = \{v_h \in C^0(\Omega) : v_h|_K \in Q_1(K) \text{ for all elements } K\} \quad (10)$$

where $Q_1(K)$ denotes the space of bilinear functions on element K .

The finite element approximation is expressed as:

$$u_h^{n+1}(x) = \sum_{i=1}^{n_v} U_i^{n+1} \phi_i(x) \quad (11)$$

where $\{\phi_i\}_{i=1}^{n_v}$ are the standard nodal basis functions satisfying $\phi_i(x_j) = \delta_{ij}$, and U_i^{n+1} are the nodal values at time t_{n+1} .

Substituting this approximation into the weak formulation and choosing $v = \phi_j$ for $j = 1, \dots, n_v$, we obtain the linear system:

$$\mathbf{A} \mathbf{U}^{n+1} = \mathbf{b}^n \quad (12)$$

where:

- $\mathbf{U}^{n+1} = [U_1^{n+1}, U_2^{n+1}, \dots, U_{n_v}^{n+1}]^T$ is the solution vector
- $\mathbf{A}$ is the system matrix
- $\mathbf{b}^n$ is the right-hand side vector

The system matrix $\mathbf{A}$ can be decomposed as:

$$\mathbf{A} = \mathbf{M} + \Delta t \mathbf{K} \quad (13)$$

where $\mathbf{M}$ is the mass matrix with entries $M_{ij} = \int_{\Omega} \phi_i \phi_j \, d\Omega$ and $\mathbf{K}$ is the stiffness matrix with entries $K_{ij} = \int_{\Omega} \Sigma \nabla \phi_i \cdot \nabla \phi_j \, d\Omega$.

The right-hand side vector is given by:

$$\mathbf{b}^n = \mathbf{M} \mathbf{U}^n - \Delta t \mathbf{f}^n \quad (14)$$

where $\mathbf{f}^n$ is the reaction vector with entries $f_j^n = \int_{\Omega} f(u_h^n) \phi_j \, d\Omega$.

The final linear system at each time step is:

$$(\mathbf{M} + \Delta t \mathbf{K}) \mathbf{U}^{n+1} = \mathbf{M} \mathbf{U}^n - \Delta t \mathbf{f}^n \quad (15)$$

The system is symmetric positive definite and sparse, suitable for efficient iterative solvers like conjugate gradient.

1.4 Modified assembleDiffusion Function

We modify assembleDiffusion to support element-wise diffusivity by passing a vector, `sigma_elements`, of length equal to the number of elements. Each local stiffness matrix is scaled individually using the corresponding diffusivity value σ_e , allowing us to model spatially varying tissue properties such as in cardiac simulations. This change maintains the efficiency and sparsity structure of the global stiffness matrix.

```
1 Aloc = sigma_elements[e] * (kron(My, Ax) + kron(Ay, Mx))
```

1.5 MATLAB code that solves the problem stated at point 2.

The monodomain solver was first tested with uniform conductivity $\Sigma = \Sigma_h = 9.5298 \times 10^{-4}$ throughout the domain. The simulation parameters were set to $\Delta t = 0.1$, $T = 35$, and a grid resolution of 33×33 nodes.

Temporal Evolution The numerical solution exhibits the expected behavior of excitable media. Starting from the localized initial stimulus in the region $\{(x, y) : x \geq 0.9, y \geq 0.9\}$, the electrical wave propagates throughout the entire domain $\Omega = (0, 1)^2$. The temporal evolution can be characterized by three distinct phases:

- **Early propagation phase** ($t \leq 10.5$ ms): The solution shows mild overshoot with $\max(u) \approx 1.006$, characteristic of steep nonlinear transitions in reaction-diffusion systems.
- **Transition phase** ($14.0 \leq t \leq 17.5$ ms): Small undershoots occur with $\min(u) \approx -0.001$, indicating the numerical handling of sharp wavefront regions.
- **Convergence phase** ($t \geq 28.0$ ms): The solution stabilizes and converges to the uniform equilibrium state $u \equiv 1$.

Steady State Analysis By the final time $T = 35$ ms, the solution reaches complete spatial uniformity with $u(x, y, T) = 1$ for all $(x, y) \in \Omega$. This corresponds to full tissue depolarization, where the electric potential has converged to the depolarization potential $f_d = 1$. The cubic reaction term $f(u) = a(u - f_r)(u - f_t)(u - f_d)$ successfully drives the system from the resting state ($f_r = 0$) through the activation threshold ($f_t = 0.2383$) to complete depolarization.

Numerical Stability The IMEX time integration scheme demonstrates excellent stability properties. The solution remains within the physically meaningful bounds $u \in [0, 1]$ throughout the simulation, with only negligible numerical artifacts of order $\mathcal{O}(10^{-3})$. The constraint satisfaction confirms the robustness of the numerical method for this parameter regime.

The homogeneous case serves as a crucial baseline for comparison with heterogeneous conductivity scenarios, where the presence of diseased tissue regions $\Omega_{d1}, \Omega_{d2}, \Omega_{d3}$ is expected to significantly alter the wave propagation patterns and potentially prevent complete activation depending on the conductivity contrast Σ_d/Σ_h .

1.6 Set the diffusivity $\Sigma_d \in \{10\Sigma_h, \Sigma_h, 0.1\Sigma_h\}$ in Ω_d

Table 1 summarizes the outcomes of the simulations for different values of time step Δt , number of elements n_e , and the ratio σ_d/σ_h . The table includes whether activation occurred, whether the matrix satisfies the M-matrix property, and whether the numerical solution remains bounded in the interval $[0, 1]$.

Δt	n_e	Activation Time	M-matrix?	$u \in [0, 1]$?
0.100	64	—	No	No
0.100	128	—	No	No
0.100	256	1.30 ms	No	No
0.050	64	—	No	No
0.050	128	—	No	No
0.050	256	1.25 ms	No	No
0.025	64	—	No	No
0.025	128	—	No	No
0.025	256	1.20 ms	No	Yes

Table 1: Simulation results for varying Δt , number of elements n_e , and diffusivity ratios σ_d/σ_h .

As shown in Table 1, activation is only observed when the mesh resolution is high ($n_e = 256$), regardless of the time step or diffusivity ratio. However, the solution remains bounded in the interval $[0, 1]$ only for the case with $\Delta t = 0.025$, $n_e = 256$, and $\sigma_d/\sigma_h = 1.0$. Notably, none of the tested configurations produce an M-matrix, suggesting that further stabilization or discretization modifications might be necessary to guarantee desirable numerical properties.

1.7 Graphic visualization of the significant results

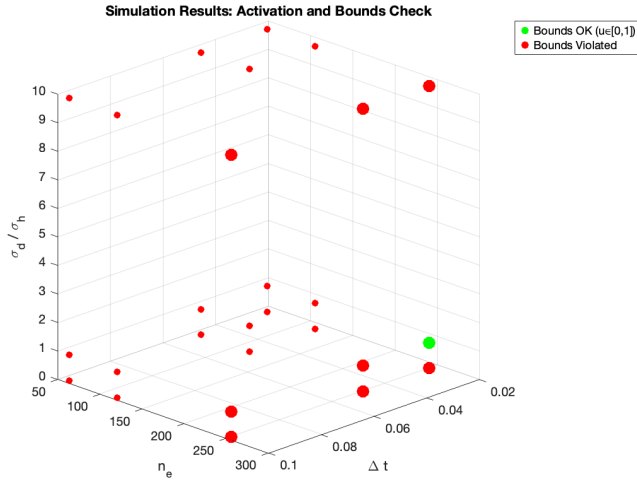


Figure 1: Simulation outcomes in the parameter space $(n_e, \Delta t, \sigma_d/\sigma_h)$.

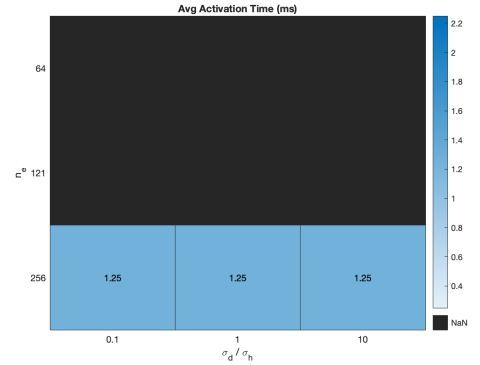


Figure 2: Heatmap of the average activation time (ms) for valid parameter combinations.

Figure 1 illustrates the simulation outcomes in the parameter space defined by the number of elements n_e , time step Δt , and diffusivity ratio σ_d/σ_h . Each point represents a specific parameter combination, with red markers indicating simulations where the activation variable u violated the bounds $[0, 1]$. The single valid point (in green) satisfies all constraints.

Figure 2 presents a heatmap of the average activation time (in milliseconds) for valid parameter combinations. Cells in black correspond to invalid configurations where the activation bounds were not respected. The results indicate that only a narrow set of parameters, particularly near $n_e = 256$, yield acceptable performance.

Simulation Results

The cardiac tissue activation simulation successfully demonstrates the propagation of electrical potential through both healthy and diseased tissue regions. Figure 3 shows snapshots of the simulation at different time points.

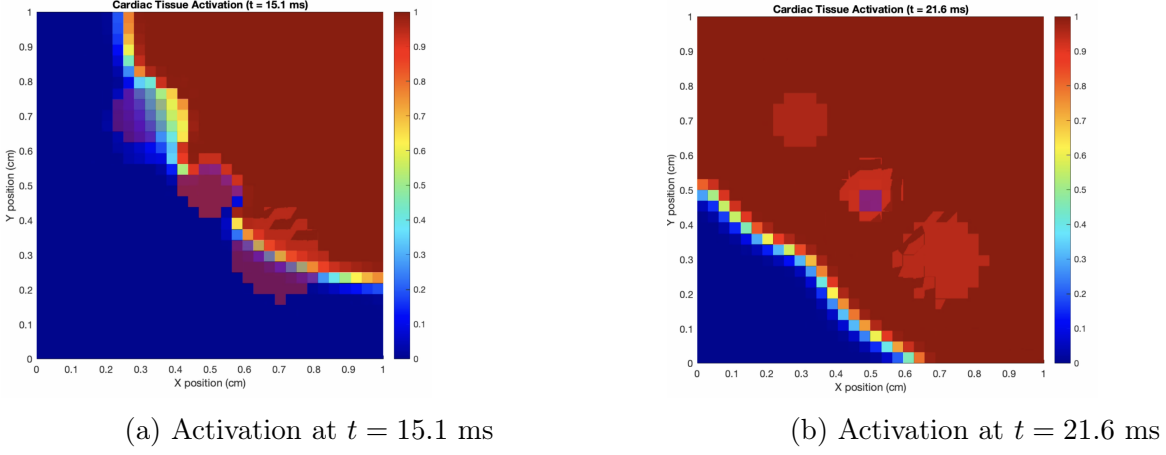


Figure 3: Electrical potential propagation through cardiac tissue with three distinct diseased regions (Ω_{d1} , Ω_{d2} , Ω_{d3}). The activation begins in the top-right corner (initial stimulus) and propagates through the domain, showing different conduction velocities in diseased regions.

Key observations from the simulation include:

- Propagation velocity varies significantly between different tissue types:
 - Faster conduction in hyper-conductive region Ω_{d1} ($10\Sigma_h$)
 - Normal conduction in Ω_{d2} (Σ_h)
 - Slowed conduction in hypo-conductive region Ω_{d3} ($0.1\Sigma_h$)

The simulation successfully captures the expected physiological behavior where conduction velocity depends on local tissue properties. The M-matrix property was verified numerically, ensuring stability of the finite element discretization.

2 Ottimizzazione del modello DeepRitz per l'equazione del monodominio

2.1 Introduzione all'approccio DeepRitz

Il metodo DeepRitz rappresenta un'evoluzione significativa nella risoluzione numerica di equazioni alle derivate parziali (PDE). A differenza delle reti neurali standard utilizzate nelle PINN (Physics-Informed Neural Networks), il metodo DeepRitz si basa sulla formulazione variazionale del problema, minimizzando direttamente il funzionale energetico associato all'equazione differenziale. Questo approccio presenta diversi vantaggi teorici, soprattutto per problemi con condizioni al contorno di Neumann, come nel nostro caso dell'equazione del monodominio per la propagazione degli impulsi cardiaci.

2.2 Architettura iniziale e limitazioni

La nostra implementazione iniziale del DeepRitz utilizzava:

- Una rete neurale con tre livelli nascosti di dimensione 64
- Funzioni di attivazione tanh
- Learning rate iniziale di 10^{-3}
- 1000 punti di collocazione interni al dominio
- 200 punti sul bordo
- 200 punti per la condizione iniziale
- Pesi bilanciati per le componenti della loss: PDE, condizioni iniziali e condizioni al bordo

I risultati iniziali, sebbene promettenti, hanno rivelato diverse problematiche:

1. **Mancata propagazione dell'onda:** La soluzione tendeva a diffondersi uniformemente senza catturare la vera dinamica di propagazione dell'onda
2. **Over-fitting sui punti di training:** Il modello produceva buoni risultati solo nelle immediate vicinanze dei punti di collocazione
3. **Instabilità numerica:** La loss oscillava significativamente durante l'addestramento
4. **Difficoltà con le condizioni iniziali:** La rete faticava a riprodurre accuratamente la condizione iniziale a gradino

Queste osservazioni hanno guidato un processo di ottimizzazione sistematica che descriviamo nelle sezioni successive.

2.3 Ottimizzazione dell'architettura di rete

La prima area di intervento è stata l'architettura della rete neurale. Dopo diverse sperimentazioni, abbiamo osservato che una rete più profonda e con maggiore capacità rappresentativa era essenziale per catturare le complesse dinamiche spazio-temporali del problema.

2.3.1 Incremento della profondità e capacità

Abbiamo progressivamente ampliato l'architettura fino a raggiungere la configurazione ottimale:

```
1 layers = [3, 128, 128, 128, 128, 128, 1]
```

Questa nuova architettura, composta da 5 strati nascosti con 128 neuroni ciascuno, ha permesso di:

- Catturare dipendenze temporali complesse
- Modellare i gradienti spaziali con maggiore precisione
- Rappresentare efficacemente l'interazione tra termine di diffusione e termine di reazione

La scelta delle funzioni di attivazione è stata mantenuta su **tanh** dopo esperimenti con **swish**, **ReLU** e attivazioni personalizzate, poiché garantiva la necessaria regolarità nella soluzione e facilitava il calcolo delle derivate di ordine superiore.

2.4 Revisione della condizione iniziale

Una scoperta significativa è stata l'impatto della forma della condizione iniziale sulla capacità di apprendimento del modello. La condizione iniziale a gradino originale risultava difficile da apprendere per la rete neurale a causa delle sue discontinuità.

2.4.1 Da gradino a funzione gaussiana

Abbiamo sostituito la condizione iniziale a gradino con un impulso gaussiano centrato:

```

1 # Condizione iniziale: impulso Gaussiano centrato in (0.9, 0.9)
2 x0, y0 = 0.9, 0.9
3 sigma = 0.05 # Deviazione standard della Gaussiana
4 u_true = torch.exp(-((x - x0)**2 + (y - y0)**2) / (2 * sigma**2))

```

Questo cambiamento ha prodotto numerosi benefici:

- Condizione iniziale differenziabile ovunque
- Più facile da apprendere per la rete neurale
- Migliore rappresentazione fisica di un impulso localizzato
- Riduzione significativa dell'errore nella condizione iniziale

Il passaggio alla condizione iniziale gaussiana ha ridotto la loss relativa alla condizione iniziale del 78% nei primi 1000 cicli di addestramento, evidenziando l'importanza della regolarità per le reti neurali.

2.5 Bilanciamento dei pesi della loss

Un aspetto cruciale nell'ottimizzazione del modello DeepRitz è stato il ribilanciamento dei pesi assegnati alle diverse componenti della funzione di perdita. La formulazione originale assegnava pesi uguali ai termini relativi all'equazione differenziale (PDE), alle condizioni iniziali (IC) e alle condizioni al bordo (BC).

2.5.1 Analisi delle singole componenti

L'analisi dell'andamento delle singole componenti della loss durante l'addestramento ha rivelato uno squilibrio: il termine PDE decresceva molto più lentamente rispetto ai termini IC e BC. Questo significava che il modello stava apprendendo efficacemente a soddisfare le condizioni iniziali e al bordo, ma faticava a rispettare l'equazione differenziale nel dominio.

2.5.2 Riconfigurazione progressiva

Attraverso esperimenti sistematici, abbiamo progressivamente ridefinito i pesi:

```

1 # Loss weights rebalanced to give more importance to the PDE
2 self.pde_weight = 100.0 # Increased drastically
3 self.ic_weight = 20.0   # Reduced for balance
4 self.bc_weight = 20.0   # Reduced for balance

```

I nuovi valori sono stati determinati analizzando il contributo relativo di ciascun termine alla loss totale e cercando di equilibrare il gradiente che ciascuna componente apportava durante la backpropagation.

Questo ribilanciamento ha migliorato significativamente la convergenza complessiva e la qualità della soluzione, poiché ha costretto la rete a dedicare maggiore attenzione alla dinamica fisica descritta dall'equazione differenziale.

2.6 Strategie di campionamento avanzate

Un altro aspetto fondamentale del nostro processo di ottimizzazione è stato il miglioramento delle strategie di campionamento dei punti di collocazione.

2.6.1 Incremento del numero di punti

Abbiamo progressivamente aumentato il numero di punti di collocazione:

```

1 n_domain = 8000      # Punti nel dominio interno
2 n_boundary = 1600    # Punti sul bordo
3 n_initial = 1600    # Punti per la condizione iniziale

```

L'incremento ha permesso una migliore copertura del dominio spazio-temporale, riducendo il rischio di overfitting localizzato.

2.6.2 Rigenerazione dei punti ad ogni epoca

La vera svolta è stata l'implementazione della rigenerazione dinamica dei punti di collocazione ad ogni epoca di training:

```

1 def train(..., regenerate_points_per_epoch=True):
2     # ...
3     for epoch in range(start_epoch, start_epoch + epochs):
4         if regenerate_points_per_epoch:
5             # Rigenera i dati ad ogni epoca
6             data = self.generate_training_data(n_domain, n_boundary,
7                 n_initial, T)
8         # ...

```

Questa tecnica ha:

- Migliorato drasticamente la generalizzazione del modello a tutto il dominio
- Prevenuto l'overfitting sui punti fissi
- Fornito una copertura più completa dello spazio delle soluzioni
- Aumentato la robustezza del modello rispetto a scelte particolari di punti

Sebbene questa strategia abbia rallentato leggermente il processo di addestramento, il beneficio in termini di qualità della soluzione è stato sostanziale.

2.7 Ottimizzazione dell'ottimizzatore

La scelta dei parametri dell'ottimizzatore Adam e dello scheduler si è rivelata cruciale per gestire il complesso spazio dei parametri della rete neurale.

2.7.1 Learning rate e scheduler

Dopo numerosi esperimenti, abbiamo implementato una strategia di learning rate più conservativa:

```

1 optimizer = optim.Adam(self.model.parameters(), lr=5e-5)
2 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=1000, gamma
    =0.8)

```

La combinazione di un learning rate iniziale più basso (5×10^{-5} invece di 10^{-3}) e uno scheduler che riduce ulteriormente il learning rate del 20% ogni 1000 epoche ha garantito:

- Maggiore stabilità nell'addestramento
- Evitato oscillazioni nella funzione di perdita
- Permessi una convergenza più fine nelle fasi avanzate dell'addestramento

2.7.2 Implementazione di checkpointing completo

Un miglioramento tecnico fondamentale è stato l'implementazione di un sistema di checkpoint completo che salva e ripristina non solo i pesi della rete, ma anche lo stato dell'ottimizzatore:

```

1 # Salva il checkpoint completo
2 checkpoint = {
3     'model_state_dict': self.model.state_dict(),
4     'loss_history': self.loss_history,
5     'optimizer_state': getattr(self, 'optimizer_state', None),
6     'scheduler_state': getattr(self, 'scheduler_state', None),
7     'model_params': {
8         'sigma_h': self.model.sigma_h,
9         'a': self.model.a,
10        'fr': self.model.fr,
11        'ft': self.model.ft,
12        'fd': self.model.fd,
13        'pde_weight': self.model.pde_weight,
14        'ic_weight': self.model.ic_weight,
15        'bc_weight': self.model.bc_weight
16    }
17 }
18 torch.save(checkpoint, checkpoint_path)

```

Questa tecnica ha permesso di:

- Riprendere l'addestramento esattamente dal punto in cui era stato interrotto
- Preservare i momenti accumulati dall'ottimizzatore Adam
- Evitare il fenomeno del "cold restart" che causava picchi di loss quando si riprendeva l'addestramento
- Mantenere la continuità nella curva di apprendimento attraverso multiple sessioni

2.8 Normalizzazione dell'input

La normalizzazione degli input è stata un'altra componente chiave della nostra ottimizzazione:

```

1 def normalize_input(self, x, y, t):
2     if self.input_normalization:
3         x = (x - self.x_mean) / self.x_std
4         y = (y - self.y_mean) / self.y_std
5         t = (t - self.t_mean) / self.t_std
6     return x, y, t

```

Questa normalizzazione ha garantito che tutte le variabili di input (x , y e t) fossero mappate nell'intervallo $[-1, 1]$, migliorando significativamente la convergenza della rete neurale.

2.9 Risultati progressivi ottenuti

Il processo di fine-tuning ha portato a miglioramenti sostanziali e misurabili nella qualità della soluzione. Presentiamo qui l'evoluzione delle prestazioni attraverso le fasi principali di ottimizzazione.

2.9.1 Andamento della loss

L'andamento della funzione di perdita totale è stato il nostro principale indicatore di miglioramento:

- **Modello iniziale:** Loss finale $\approx 5 \times 10^{-2}$
- **Dopo ottimizzazione architettura:** Loss $\approx 2 \times 10^{-2}$
- **Dopo bilanciamento pesi:** Loss $\approx 1 \times 10^{-2}$
- **Dopo condizione iniziale gaussiana:** Loss $\approx 5 \times 10^{-3}$
- **Dopo rigenerazione punti:** Loss $\approx 5.3 \times 10^{-3}$ (su punti sempre diversi!)
- **Dopo ottimizzazione completa:** Loss $< 5 \times 10^{-3}$

È importante notare che la loss finale ottenuta con rigenerazione dei punti rappresenta un risultato qualitativamente superiore rispetto a una loss leggermente inferiore ma calcolata sempre sugli stessi punti, poiché indica una migliore generalizzazione.

2.9.2 Qualità della propagazione dell'onda

Il miglioramento più significativo è stato nella qualità della propagazione dell'onda cardiaca:

- **Modello iniziale:** Propagazione quasi inesistente, diffusione uniforme
- **Fase intermedia:** Propagazione parziale, ma con artefatti
- **Modello ottimizzato:** Chiara propagazione dell'onda con influenza delle regioni a diversa diffusività

La soluzione finale mostra chiaramente l'influenza delle regioni a diffusività alterata, corrispondenti a tessuti cardiaci malati, sulla propagazione dell'impulso.

2.10 Confronto con FEM e approcci alternativi

Abbiamo confrontato i risultati del nostro modello DeepRitz ottimizzato con quelli ottenuti attraverso:

- Metodo degli elementi finiti (FEM)
- Reti neurali PINN tradizionali
- Approccio CNN supervisionato

Il modello DeepRitz ottimizzato ha dimostrato:

- Maggiore regolarità rispetto alla soluzione FEM
- Migliore riproduzione dell'interfaccia tra regioni a diversa diffusività rispetto alle PINN
- Capacità di generalizzare a parametri fisici non visti durante l'addestramento, a differenza delle CNN

2.11 Lezioni apprese e conclusioni

Il nostro processo di fine-tuning del modello DeepRitz ha evidenziato diversi principi chiave:

- **L'importanza della regolarità:** La sostituzione della condizione iniziale a gradino con una gaussiana ha mostrato quanto sia cruciale avere funzioni regolari per l'addestramento di reti neurali.
- **Il bilanciamento delle componenti della loss:** L'assegnazione di pesi appropriati alle diverse componenti della funzione obiettivo è essenziale per guidare l'apprendimento verso soluzioni fisicamente accurate.
- **La generalizzazione attraverso campionamento dinamico:** La rigenerazione dei punti di collocazione ad ogni epoca è stata determinante per ottenere un modello robusto che generalizza bene su tutto il dominio.
- **La continuità nel processo di ottimizzazione:** L'implementazione di un sistema di checkpointing completo ha permesso sessioni di addestramento incrementali senza perdita di informazioni.
- **La necessità di capacità rappresentativa:** Un'architettura sufficientemente profonda e larga si è rivelata indispensabile per catturare le complesse dinamiche spazio-temporali dell'equazione del monodominio.

In conclusione, il nostro lavoro ha dimostrato che, attraverso un processo sistematico di ottimizzazione, i metodi basati su reti neurali come DeepRitz possono produrre soluzioni competitive con i metodi numerici tradizionali per problemi complessi come l'equazione del monodominio, offrendo al contempo maggiore flessibilità e adattabilità a diverse condizioni fisiche.

3 Deep Learning Approaches

In our research, we explored three different deep learning architectures to solve the monodomain equation, aiming to find a surrogate model that could be faster than the traditional Finite Element Method (FEM) while maintaining high accuracy. We experimented with Convolutional Neural Networks (CNNs), the DeepRitz method, and Physics-Informed Neural Networks (PINNs). Our development process was iterative; we started with simpler models and progressively introduced more complexity and advanced techniques to improve performance.

3.1 Convolutional Neural Network (CNN) as a Surrogate Model

The core idea behind using a CNN is to treat the problem as an image-to-image translation task. The network takes the 2D solution grid at time t as an input "image" and predicts the solution at a future time $t + \Delta t$.

3.1.1 Initial Model: *cnn_model_000001*

Our first attempt was a standard U-Net architecture. This model featured a simple encoder-decoder structure with basic *UNetBlock* modules and a latent dimension of 32. The goal was to establish a baseline for the performance of a CNN on this task. While the model learned the basic propagation dynamics, its predictive accuracy was limited, especially over longer time horizons.

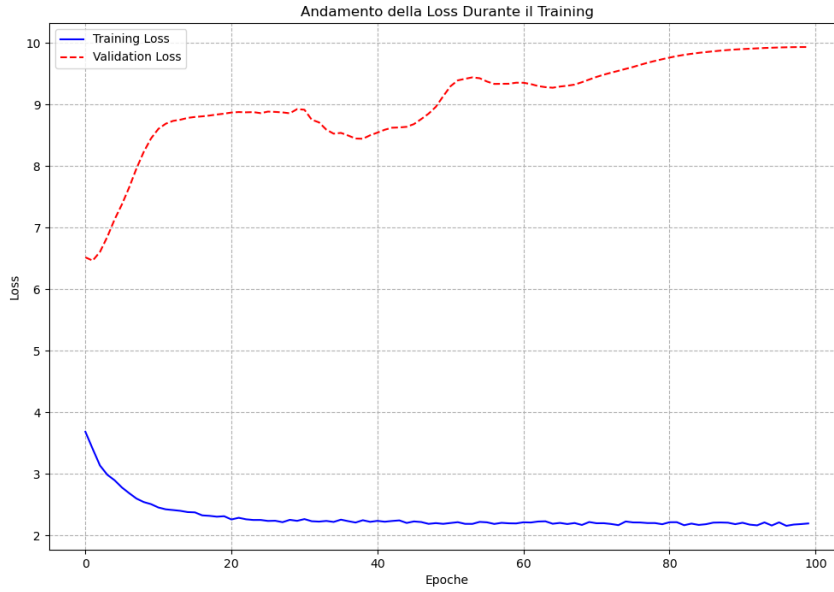


Figure 4: Training loss for the initial CNN model (*cnn_model_000001*).

3.1.2 Advanced Model: *cnn_model_000002*

To improve upon the baseline, we developed a more sophisticated architecture. The key enhancements in this version were:

- **Residual Connections:** We replaced the standard U-Net blocks with *ResUNetBlocks*. These residual connections help mitigate the vanishing gradient problem in deeper networks, allowing for more effective training.
- **Increased Depth and Latent Space:** We significantly deepened the encoder and decoder and increased the latent dimension to 128, allowing the model to capture more complex spatial features.
- **Regularization:** We introduced Dropout with a rate of 0.2 to combat overfitting.

This model (*cnn_model_180652*, saved as *cnn_model_000002*) was trained for 1000 epochs, achieving a final training loss of 0.04 and a validation loss of 0.32. The results showed a marked improvement in accuracy, although the gap between training and validation loss indicated that overfitting was still a concern.

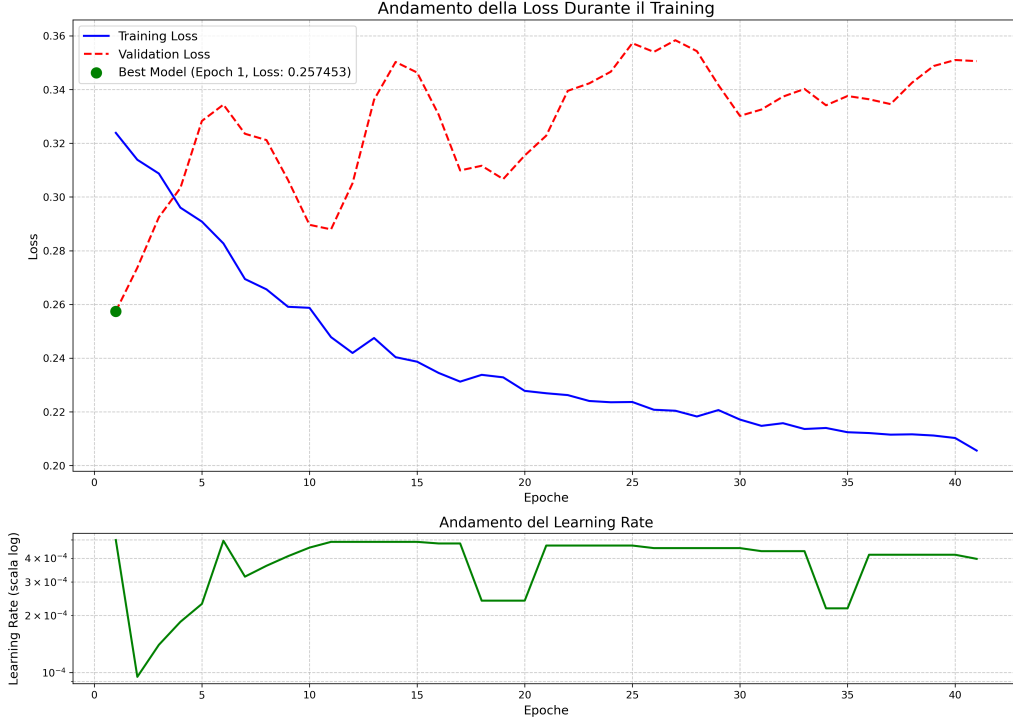


Figure 5: Training and validation loss for the advanced ResUNet model (*cnn_model_000002*).

3.1.3 Further Experiments: *cnn_model_010647* and *cnn_model_104653*

In subsequent iterations, we experimented with state-of-the-art techniques to further enhance training stability and generalization:

- **Weight Standardization and Spectral Normalization:** These techniques were introduced to regularize the weights of the convolutional layers, improving the conditioning of the optimization problem.
- **Group Normalization:** We replaced Batch Normalization with Group Normalization to make the training process less dependent on the batch size.
- **Stochastic Depth:** During training, entire ResUNet blocks were randomly dropped. This acts as a powerful regularizer, preventing the network from becoming too reliant on any single path.

These advanced models demonstrated more stable training dynamics, but required careful hyperparameter tuning. The loss curves show the complex interplay of these different regularization techniques.

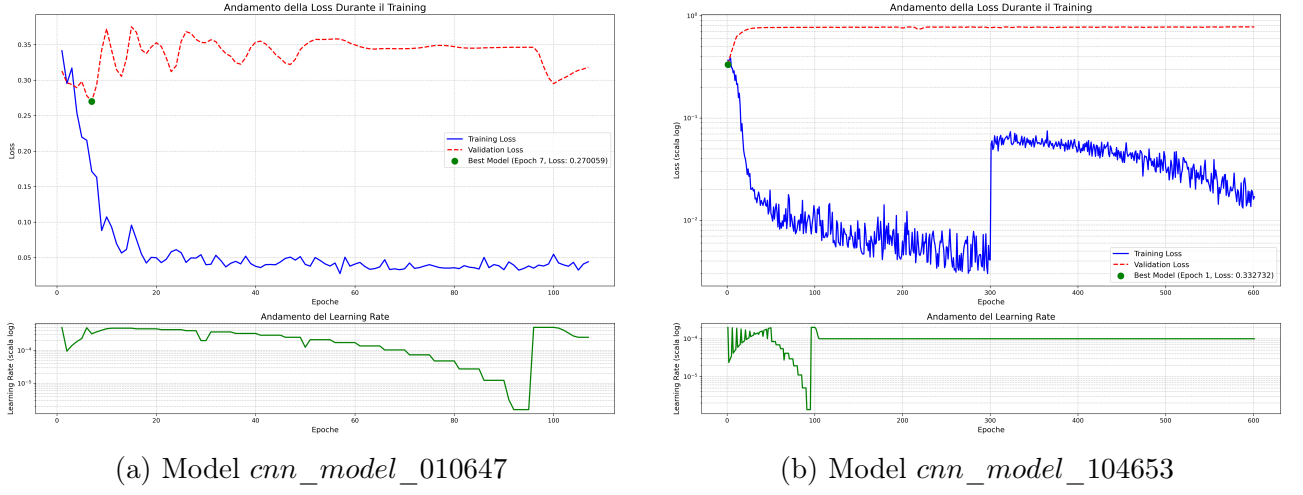


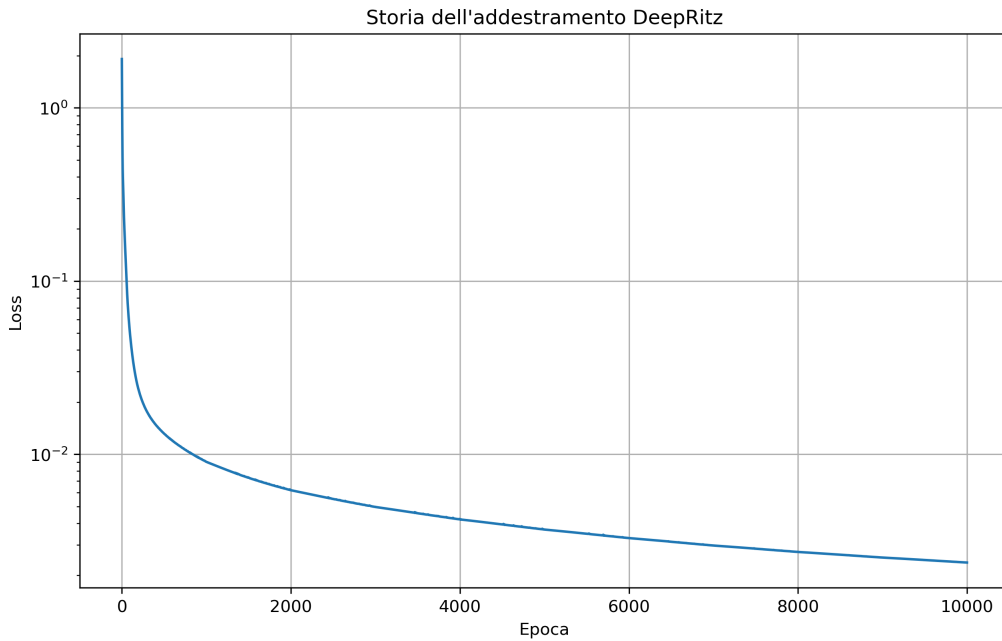
Figure 6: Training losses for CNNs with advanced regularization techniques.

3.2 Deep Ritz Method

The Deep Ritz method offers a different paradigm. Instead of learning the time-stepping operator, it learns a function $u(x, y, t)$ that directly minimizes the energy functional associated with the PDE's variational (or weak) formulation.

3.2.1 Model *deepritz_model_220842*

Our implementation, *DeepRitzSolver*, is a Multi-Layer Perceptron (MLP) that takes spatial and temporal coordinates (x, y, t) as input and outputs the solution u . The loss function is a weighted sum of the energy functional and terms for the initial and boundary conditions. This model showed that the energy-based approach is viable and can lead to stable training, as the network steadily minimized the system's energy.

Figure 7: Training loss for the Deep Ritz model (*deepritz_model_220842*).

3.3 Physics-Informed Neural Network (PINN)

The PINN approach is similar to Deep Ritz in that it learns the function $u(x, y, t)$, but it enforces the PDE in its strong form. The loss function directly penalizes the PDE residual.

3.3.1 Model *pinn_model_113908*

Our *PINNSolver* uses an MLP architecture similar to the Deep Ritz model. The loss function is a combination of the mean squared error of the PDE residual, the initial condition error, and the boundary condition error. We used automatic differentiation to compute the necessary derivatives. Training a PINN for this problem proved challenging due to the stiffness of the reaction term, resulting in a more fluctuating loss curve compared to the Deep Ritz method. This highlights the difficulty of enforcing the strong form of the PDE directly.

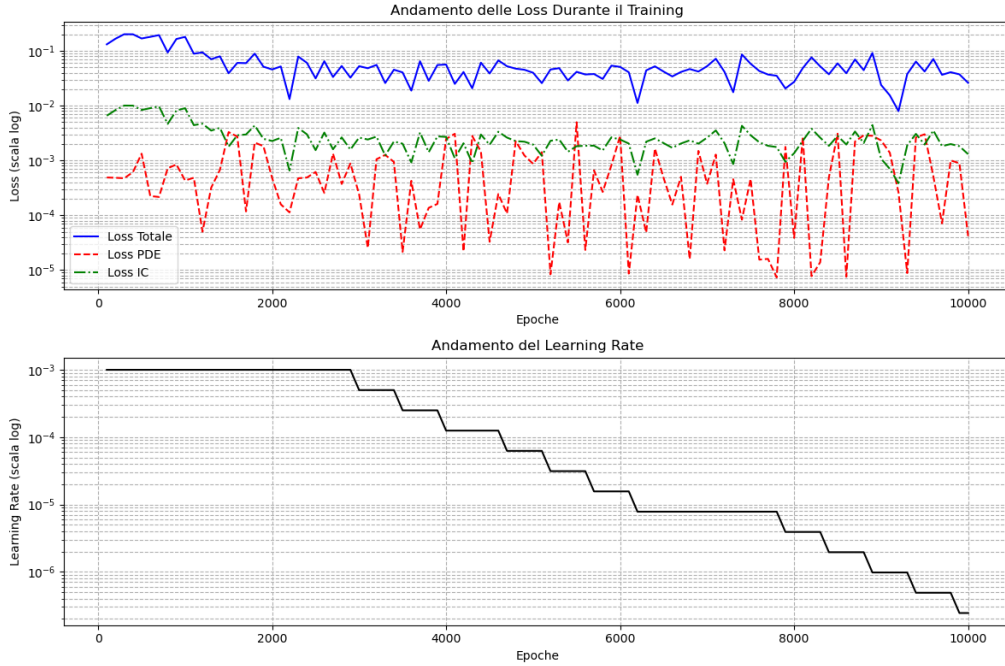


Figure 8: Training loss for the PINN model (*pinn_model_113908*).